

Technical Report

AI-Assisted Free-Space Optical Communication (FSOC) Beacon Tracking System

Coarse Pointing, Acquisition, and Tracking (PAT) Simulator

Version 1.0 | September 2026

Abstract

Free-space optical communication (FSOC) offers multi-Gbps throughput, but its narrow, highly coherent beams demand sub-milliradian alignment between terminals. This report documents the design, implementation, and validation of a software simulator for *closed-loop coarse Pointing, Acquisition, and Tracking* (PAT) using camera-based beacon tracking. The system combines classical computer vision with a YOLO-family neural detector, fuses both outputs, and feeds a Kalman/Particle adaptive hybrid state estimator that drives a dual-axis PID gimbal controller with predictive lead-ahead compensation. A ten-plus-state acquisition finite state machine, a dynamic Safe-FOV risk engine, and a five-factor Fine-PAT handoff readiness evaluator complete the control stack. Realistic disturbances (platform vibration, atmospheric turbulence, sensor noise, blur, and line-of-sight occlusion) are injected to stress the pipeline. In benchmark scenarios, the full pipeline (Algorithm E) reduces pointing RMSE by a factor of 2.4-2.7 relative to a classical-detector plus direct-PID baseline. The report also presents the mathematical formulation of every estimation and control component, the test methodology, and a roadmap for future work.

Contents

1	Problem Understanding	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Key Challenges Addressed	3
2	System Architecture	3
2.1	High-Level Architecture	3
2.2	Closed-Loop Data Flow	4
2.3	Repository Layout	4
3	Description of Software Modules	5
3.1	Simulation Module (<code>simulation/</code>)	5
3.2	Vision Module (<code>vision/</code>)	6
3.3	Tracking Module (<code>tracking/</code>)	6
3.4	Control Module (<code>control/</code>)	6
3.5	Disturbance Module (<code>disturbances/</code>)	7
3.6	Evaluation Module (<code>evaluation/</code>)	7
3.7	Application Module (<code>app/</code>)	7
4	Vision and Detection Methods	7
4.1	Classical Beacon Detector	7
4.2	YOLO Object Detection	8

4.3	Multi-Modal Fusion	8
4.4	Graceful Degradation	8
5	Tracking and Estimation Methods	9
5.1	Kalman Filter	9
5.2	Particle Filter	9
5.3	Optical Severity Score	10
5.4	Adaptive Hybrid Estimator	10
5.5	Acquisition Finite State Machine	10
6	Control Methods	10
6.1	PID Controller with Dead-Zone and Anti-Windup	10
6.2	Latency Compensation	11
6.3	Search Strategies	12
6.4	Dynamic Safe-FOV and Risk Assessment	12
6.5	Fine-PAT Handoff Readiness	12
7	Test Methodology	12
7.1	Unit and Integration Tests	12
7.2	Scenario-Based Integration Testing	12
7.3	Multi-Algorithm Benchmarking	12
7.4	Adversarial Stress Testing (PAT-RED)	12
8	Performance Analysis	13
8.1	Demonstration-Tour Results	13
8.2	Per-Act Breakdown	13
8.3	Algorithm Comparison	13
8.4	Key Observations	14
8.5	Limitations	14
9	Future Improvements	14
9.1	Short Term	14
9.2	Medium Term	15
9.3	Long Term	15
10	Conclusion	16
A	Notation	16
B	Glossary	16

1 Problem Understanding

1.1 Background

Free-Space Optical Communication (FSOC) transmits data using modulated laser beams propagating through the atmosphere or vacuum. Compared with radio-frequency (RF) links, FSOC offers orders-of-magnitude higher bandwidth (multi-Gbps), inherent immunity to electromagnetic interference, a smaller system footprint, and licence-free spectrum. These properties make it attractive for satellite-to-ground downlinks, UAV relay networks, inter-satellite links, and last-mile terrestrial backhaul.

The price of this performance is extreme spatial coherence: beam divergence angles are typically tens to hundreds of *micro-radians*. For a pointing error θ_e and link range R , the beam centroid is displaced at the receiver by

$$\Delta x \approx R \theta_e. \quad (1)$$

With $\theta_e = 10 \mu\text{rad}$ and $R = 1000 \text{ km}$, Eq. (1) gives $\Delta x = 10 \text{ m}$, far larger than a typical optical aperture. Establishing and sustaining a link therefore requires an actively controlled **PAT** subsystem that keeps the transmit and receive telescopes mutually aligned in real time.

1.2 Problem Statement

The objective is to design, implement, and validate a software system that demonstrates **closed-loop coarse PAT** for an FSOC terminal using camera-based beacon tracking. The system must:

1. **Acquire** an optical beacon of uncertain initial angular position by executing systematic scan patterns across the Field of Regard (FOR).
2. **Detect** the beacon in a camera sensor frame using classical computer vision and AI-based object detection, fusing their outputs for high reliability.
3. **Estimate** the beacon's kinematic state (position, velocity) with recursive Bayesian filters (Kalman and Particle) that are robust to scintillation and measurement dropouts.
4. **Track** the beacon with a closed-loop dual-axis PID gimbal controller featuring dead-zone locking, predictive lead-ahead control, and anti-windup protection.
5. **Survive** platform vibration, atmospheric turbulence, sensor noise, motion blur, and temporary line-of-sight (LOS) occlusion without losing track.
6. **Evaluate** readiness for Coarse-to-Fine PAT handoff by confirming that pointing error, jitter, and angular velocity satisfy the thresholds required by Fine-PAT subsystems such as Fast Steering Mirrors (FSMs).

1.3 Key Challenges Addressed

2 System Architecture

2.1 High-Level Architecture

The system follows a modular, layered architecture with a clear separation of concerns (Fig. 1). Each layer exposes a narrow interface to its neighbours, so any subsystem (e.g. the detector or the estimator) can be swapped without touching the rest of the pipeline.

Challenge	Impact on PAT	Solution in this system
Unknown initial beacon position	Cannot begin tracking	Acquisition FSM with Archimedean spiral and raster scan
Gimbal motor latency (~1–3 frames)	Phase-lagged control response	Kalman-based multi-step lead-ahead prediction
Atmospheric turbulence (scintillation)	Non-Gaussian beam wander and intensity fading	Adaptive hybrid Kalman $\leftrightarrow$ Particle filter with severity switching
Temporary LOS occlusion	Total loss of optical signal	Zero-measurement Kalman dead-reckoning (coasting)
Platform vibration and jitter	High-frequency random pointing offsets	Vibration rejection in PID with filtered derivative
Optical clutter and distractors	False detections	Multi-criteria candidate scoring and temporal association
Fine-PAT handoff precision	Premature handoff causes link drop	5-factor weighted Handoff Readiness Engine

Table 1: Principal PAT challenges and the corresponding design responses.

2.2 Closed-Loop Data Flow

At each simulation frame the pipeline executes the following steps (Fig. 2):

1. **Physics update:** the environment advances all beacon trajectories.
2. **Optical disturbance injection:** platform vibration offsets are added to the camera LOS.
3. **Sensor rendering:** the virtual camera projects all beacons into a synthetic frame.
4. **Visual disturbance application:** turbulence warping, motion blur, sensor noise, and frame drops.
5. **Vision detection:** classical and AI detectors run on the disturbed frame; outputs are fused.
6. **State estimation:** the tracker updates the Kalman/Particle filter and the acquisition FSM.
7. **Control actuation:** the PID controller converts estimated angular error into rate commands.
8. **Gimbal kinematics:** the camera integrates velocity commands and enforces mechanical limits.
9. **Ground-truth evaluation:** pointing error is computed against the known beacon position.
10. **HUD rendering:** overlays and telemetry are drawn for display or video recording.

2.3 Repository Layout

```

fsoc-pat/
|-- app/ main.py (CLI runner); gui/ (PySide6 dashboard, plots, widgets)
|-- simulation/ camera, beacon, environment, trajectory, coordinate_system, hud
|-- vision/ classical_detector, yolo_detector, fusion, detection_types, preprocessing
|-- tracking/ kalman, particle, hybrid, severity, state_machine, tracker
|-- control/ pid, camera_controller, search_strategy, safe_fov, handoff, latency
|-- disturbances/ manager, noise, vibration, turbulence, blur, occluder

```

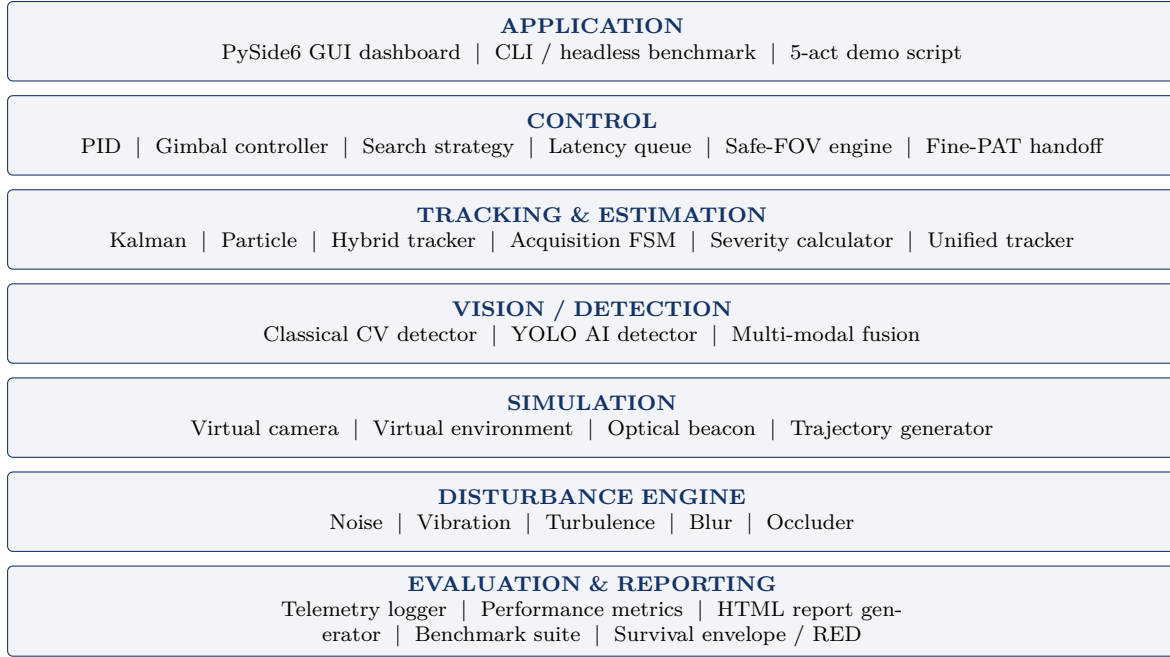


Figure 1: Layered software architecture of the PAT simulator.

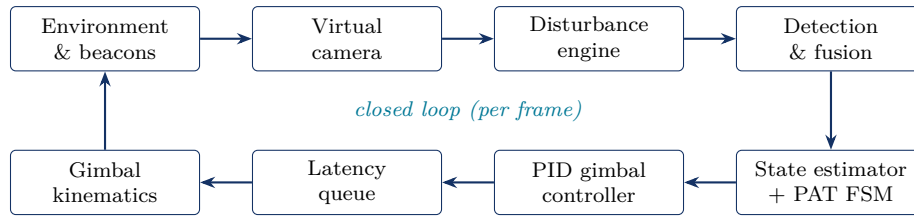


Figure 2: Simplified closed-loop signal path. The gimbal’s new boresight feeds back into the next rendered frame.

```

|-- evaluation/ metrics, logger, report_generator, benchmark, survival_envelope
|-- configs/ default, easy, uav, satellite, extreme (.json)
|-- models/ AI model weights
|-- tests/ automated unit and integration tests
|-- scripts/ demo_simulation.py (5-act guided tour)
|-- reports/ generated telemetry, reports, videos
|-- requirements.txt, demo.py

```

3 Description of Software Modules

3.1 Simulation Module (`simulation/`)

Purpose: provides the virtual physical world in which the PAT system operates.

- **VirtualCamera** (`camera.py`) simulates a two-axis pan-tilt gimbal camera. It models field of view (FOV), gnomonic pixel projection, gimbal kinematics with mechanical hard-stop limits, angular-rate saturation, and platform-jitter injection. The camera integrates the commanded velocity each time step and enforces physical constraints.
- **Beacon** (`beacon.py`) represents a target or distractor with configurable intensity, Point Spread Function (PSF) radius, colour, trajectory binding, and temporal blinking (frequency and duty cycle). A `history` deque stores past positions for trail rendering.

- **Trajectory classes** (`trajectory.py`) generate motion profiles: **LinearTrajectory** (constant velocity, optional constant acceleration), **CircularTrajectory** (orbital motion), **SinusoidalTrajectory** (horizontal drift with vertical flutter), and **ManeuverTrajectory** (pseudo-random jinking with cubic-spline acceleration changes).
- **VirtualEnvironment** (`environment.py`) manages the angular world boundaries, a persistent starfield, and all beacons. Background stars are world-fixed and drift correctly as the camera pans. It also renders a panoramic tactical overview map with trajectory trails and the live camera-FOV footprint.
- **HUD renderer** (`hud.py`) draws the boresight crosshair, lock dead-zone reticle, detection box, ground-truth marker, Kalman predicted-intercept point, Safe-FOV boundary, and a telemetry panel with state badges, error read-outs, filter-mode indicator, and handoff bar.

3.2 Vision Module (`vision/`)

Purpose: extracts beacon candidates from raw sensor frames using parallel detection engines.

- **ClassicalBeaconDetector** is a physics-informed multi-stage pipeline (Section 4.1).
- **YOLODetector** supports OpenCV-DNN (ONNX) and Ultralytics (PyTorch) backends, with automatic letterboxing, non-maximum suppression (NMS), and a no-op fallback when no model file is present (Section 4.2).
- **DetectionFusion** associates and merges candidates from both detectors into unified, confidence-scored detections (Section 4.3).

3.3 Tracking Module (`tracking/`)

Purpose: Bayesian state estimation, adaptive filter selection, and an aerospace-style acquisition state machine.

- **KalmanTracker:** 4-state linear Kalman filter over $[az, el, v_{az}, v_{el}]$ with DWPA process noise, chi-square innovation gating, and multi-step lead-ahead prediction.
- **ParticleTracker:** 300-particle bootstrap SIR filter with a heavy-tailed likelihood and systematic resampling when $N_{\text{eff}} < N/2$.
- **HybridPATTracker:** switches between the two filters from a real-time severity score, with hysteresis (PF at $S > 0.55$; KF recovery at $S < 0.30$ for 5 calm frames).
- **OpticalSeverityCalculator:** normalised severity $S \in [0, 1]$ from four weighted components.
- **TrackingStateMachine:** acquisition lifecycle governed by hysteresis counters, M-of-N confirmation windows, and timeouts.
- **PATTracker:** top-level coordinator of the filters, FSM, Safe-FOV engine, handoff engine, and search-strategy selection; emits a **TrackerOutput** dataclass consumed by control and visualisation.

3.4 Control Module (`control/`)

Purpose: translate tracking-error estimates into gimbal actuator commands.

- **PIDController:** discrete single-axis controller with smooth dead-zone, integrator anti-windup, filtered derivative, and output saturation (Section 6.1).
- **CameraGimbalController:** two independent PID axes (pan, tilt) plus a **ControlDelayQueue** introducing 1–3 frames of transport latency.

- **Search strategies:** `SpiralScan` (Archimedean spiral), `RasterScan` (boustrophedon sweep), and `PredictiveSearch` (expanding spiral centred on the Kalman-extrapolated last-known position).
- **DynamicSafeFOVEngine:** adaptive safe-tracking margins based on target velocity, jitter amplitude, and boundary proximity; outputs a risk state (`STABLE`, `WARNING`, `LOCK AT RISK`, `CRITICAL`) and a pixel-space box for the HUD.
- **FinePATHandoffEngine:** five-criterion handoff readiness evaluator (Section 6.5).
- **ControlDelayQueue:** FIFO buffer modelling motor-bus and actuator lag.

3.5 Disturbance Module (`disturbances/`)

Component	Model
<code>SensorNoise</code>	Gaussian noise with configurable σ , plus salt-and-pepper pixel defects.
<code>PlatformVibration</code>	Multi-harmonic sinusoidal angular jitter injected into the camera LOS, modelling structural resonance of airborne/spaceborne platforms.
<code>AtmosphericTurbulence</code>	Time-varying 2D displacement field (Kolmogorov-inspired beam wander) applied with OpenCV <code>remap</code> .
<code>MotionBlur</code> / <code>DefocusBlur</code>	Directional kernel aligned with gimbal slew, and Gaussian defocus.
<code>DynamicOccluder</code> / <code>OccluderManager</code>	Moving obstacles in angular space that can fully block the beacon, with soft Gaussian edge attenuation.
<code>DisturbanceManager</code>	Central hub coordinating all sources with normalised 0–100 % sliders.

Table 2: Disturbance models.

3.6 Evaluation Module (`evaluation/`)

- **PerformanceMetricsCalculator:** ground-truth-validated KPIs (mean error, RMSE, P50/P95/P99, lock-retention rate, acquisition time, detector latency, loss-event count).
- **TelemetryLogger:** per-frame CSV and JSON streaming.
- **PerformanceReportGenerator:** self-contained HTML reports with embedded Matplotlib charts and KPI scorecards.
- **Benchmark suite:** compares five algorithm configurations across scenario presets.
- **SurvivalEnvelope:** 2D capability matrices (target velocity vs. turbulence) and adversarial stress testing.

3.7 Application Module (`app/`)

The CLI (`main.py`) offers `-headless` (benchmark) and `-preview` (live OpenCV viewport) modes. The PySide6 dashboard provides a multi-threaded simulation worker, live camera viewport, PyQtGraph strip-charts, interactive disturbance sliders, PID tuning panel, manual jog controls, and a scenario preset selector.

4 Vision and Detection Methods

4.1 Classical Beacon Detector

The classical pipeline proceeds through: (1) grayscale conversion and Gaussian smoothing; (2) fixed or adaptive binary thresholding; (3) morphological opening with an elliptical kernel; (4)

connected-component contour extraction; (5) multi-criteria filtering on area, circularity, aspect ratio, and peak intensity; (6) weighted confidence scoring; and (7) temporal association with the last known position.

Circularity of a contour with area A and perimeter P is

$$\kappa = \frac{4\pi A}{P^2} \in (0, 1], \quad (2)$$

with $\kappa = 1$ for a perfect disc. The candidate confidence is

$$C_{cl} = 0.45 \kappa + 0.35 b + 0.20 r_a, \quad (3)$$

where b is normalised peak brightness and r_a is an aspect-ratio score. For temporal association, a candidate at pixel position $\mathbf{p}$ receives a Gaussian proximity weight relative to the last known position $\mathbf{p}_{\text{last}}$:

$$w_t(\mathbf{p}) = \exp\left(-\frac{\|\mathbf{p} - \mathbf{p}_{\text{last}}\|^2}{2\sigma_t^2}\right). \quad (4)$$

4.2 YOLO Object Detection

The system integrates a YOLOv8-family network with two interchangeable backends:

1. **OpenCV DNN (ONNX).** The input frame is letterboxed to 640×640 , normalised to $[0, 1]$, and passed through the network. The output tensor (shape $[1, 5, 8400]$) is parsed for box coordinates and class confidence, and NMS (default IoU threshold 0.45) removes duplicates.
2. **Ultralytics PyTorch.** Direct inference for `.pt` models, enabling training integration.

If no model file exists, the detector degrades to a no-op and the pipeline runs classical-only (Section 4.4).

4.3 Multi-Modal Fusion

DetectionFusion operates in three phases:

1. **Spatial association:** nearest-neighbour matching of classical and AI detections within a 30-pixel radius.
2. **Confidence-weighted centroid averaging:** for a matched pair with centroids $\mathbf{c}_{cl}, \mathbf{c}_{ai}$ and confidences C_{cl}, C_{ai} ,

$$\mathbf{c}_f = \frac{C_{cl} \mathbf{c}_{cl} + C_{ai} \mathbf{c}_{ai}}{C_{cl} + C_{ai}}, \quad (5)$$

which yields sub-pixel precision.

3. **Multi-factor scoring:**

$$C_f = 0.35 C_{ai} + 0.25 b + 0.20 \kappa + 0.20 w_t, \quad (6)$$

with a 20% penalty applied to AI-only (unmatched) candidates.

4.4 Graceful Degradation

If no AI model file is present, the system falls back automatically to classical-only detection with dynamic weight redistribution over the remaining terms of Eq. (6). The simulator thus remains fully operational without any AI model while benefiting from improved robustness when one is available.

5 Tracking and Estimation Methods

5.1 Kalman Filter

The core estimator is a discrete-time linear Kalman filter with state $\mathbf{x} = [\text{az}, \text{el}, v_{\text{az}}, v_{\text{el}}]^\top$ and a constant-velocity model $\mathbf{x}_k = \mathbf{F}\mathbf{x}_{k-1} + \mathbf{w}_k$:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \quad (7)$$

Process noise (DWPA). Under the Discrete Wiener Process Acceleration model, a piecewise-constant random acceleration with spectral density q enters through $\mathbf{\Gamma} = [\Delta t^2/2, \Delta t]^\top$ per axis, giving per-axis blocks

$$\mathbf{Q}_{\text{axis}} = q \mathbf{\Gamma} \mathbf{\Gamma}^\top = q \begin{bmatrix} \Delta t^4/4 & \Delta t^3/2 \\ \Delta t^3/2 & \Delta t^2 \end{bmatrix}, \quad (8)$$

which couples position and velocity uncertainty in a physically consistent manner.

Predict / update.

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F} \hat{\mathbf{x}}_{k-1|k-1}, \quad \mathbf{P}_{k|k-1} = \mathbf{F} \mathbf{P}_{k-1|k-1} \mathbf{F}^\top + \mathbf{Q}, \quad (9)$$

$$\mathbf{y}_k = \mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_{k|k-1}, \quad \mathbf{S}_k = \mathbf{H} \mathbf{P}_{k|k-1} \mathbf{H}^\top + \mathbf{R}, \quad (10)$$

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}^\top \mathbf{S}_k^{-1}, \quad \hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \mathbf{y}_k. \quad (11)$$

Innovation gating. The normalised innovation squared

$$d^2 = \mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y} \quad (12)$$

is compared against a chi-square threshold; measurements exceeding the 4σ gate are rejected, protecting the filter from false detections.

Numerical stability. The covariance update uses the Joseph form, which preserves positive semi-definiteness:

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K} \mathbf{H}) \mathbf{P}_{k|k-1} (\mathbf{I} - \mathbf{K} \mathbf{H})^\top + \mathbf{K} \mathbf{R} \mathbf{K}^\top. \quad (13)$$

Lead-ahead prediction. To compensate for end-to-end loop latency, `predict_ahead(steps, dt)` extrapolates the position H frames ahead:

$$\hat{\mathbf{p}}_{k+H} = \hat{\mathbf{p}}_k + \hat{\mathbf{v}}_k (H \Delta t). \quad (14)$$

Algorithm E in the benchmark uses $H = 5$.

5.2 Particle Filter

Under severe scintillation ($C_n^2 \geq 10^{-13} \text{ m}^{-2/3}$) the measurement-noise distribution becomes strongly non-Gaussian. The particle filter represents the posterior with $N = 300$ weighted particles $\{\mathbf{x}^{(i)}, w^{(i)}\}$, $\mathbf{x}^{(i)} = [\text{az}, \text{el}, v_{\text{az}}, v_{\text{el}}]$.

- **Prediction:** each particle is propagated with the kinematic model plus independent Gaussian process noise.

- **Update:** weights are multiplied by a heavy-tailed (Student- t) likelihood centred on the measurement,

$$w_k^{(i)} \propto w_{k-1}^{(i)} \left(1 + \frac{1}{\nu} \mathbf{y}^{(i)\top} \mathbf{R}^{-1} \mathbf{y}^{(i)} \right)^{-\frac{\nu+m}{2}}, \quad (15)$$

where ν is the degrees of freedom and m the measurement dimension. Heavy tails limit the influence of scintillation-induced outliers.

- **Resampling:** systematic resampling is triggered when

$$N_{\text{eff}} = \frac{1}{\sum_i (w^{(i)})^2} < \frac{N}{2}, \quad (16)$$

preventing particle degeneracy.

5.3 Optical Severity Score

The hybrid estimator is driven by a normalised severity score

$$S = 0.40 s_{\text{conf}} + 0.25 s_{\text{err}} + 0.25 s_{\text{turb}} + 0.10 s_{\text{miss}}, \quad S \in [0, 1], \quad (17)$$

where s_{conf} is confidence dropout, s_{err} normalised tracking error, s_{turb} atmospheric turbulence intensity, and s_{miss} the count of consecutive detection misses (each term normalised to $[0, 1]$).

5.4 Adaptive Hybrid Estimator

- $S > 0.55 \Rightarrow$ **Particle Filter** (robust to non-Gaussian, multi-modal noise).
- $S < 0.30$ sustained for 5 frames $\Rightarrow$ **Kalman Filter** (efficient and optimal under Gaussian noise).

The asymmetric thresholds and dwell time provide hysteresis that prevents mode chatter. State is transferred bidirectionally: **KF** $\rightarrow$ **PF**: the particle cloud is seeded from the Kalman covariance ellipse; **PF** $\rightarrow$ **KF**: the Kalman state is re-initialised from the weighted particle mean.

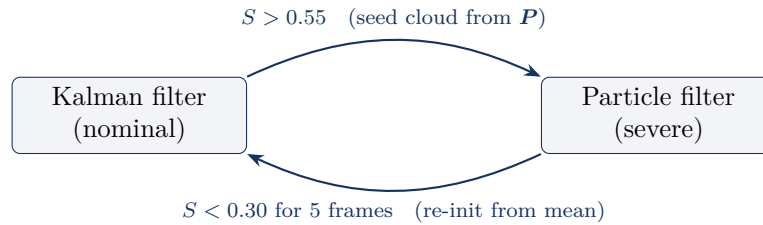


Figure 3: Hysteretic switching between the Kalman and Particle estimators.

5.5 Acquisition Finite State Machine

The PAT lifecycle is managed by an aerospace-style FSM (Fig. 4, Table 3). Transitions are guarded by hysteresis counters, M-of-N confirmation windows, and timeout thresholds so that the machine does not chatter between, e.g., tracking and lost.

6 Control Methods

6.1 PID Controller with Dead-Zone and Anti-Windup

Each gimbal axis uses a discrete PID controller. With error e_k and dead-zone half-width δ , the effective error is

$$\tilde{e}_k = \begin{cases} 0, & |e_k| \leq \delta \\ e_k - \text{sgn}(e_k) \delta, & |e_k| > \delta \end{cases} \quad (18)$$

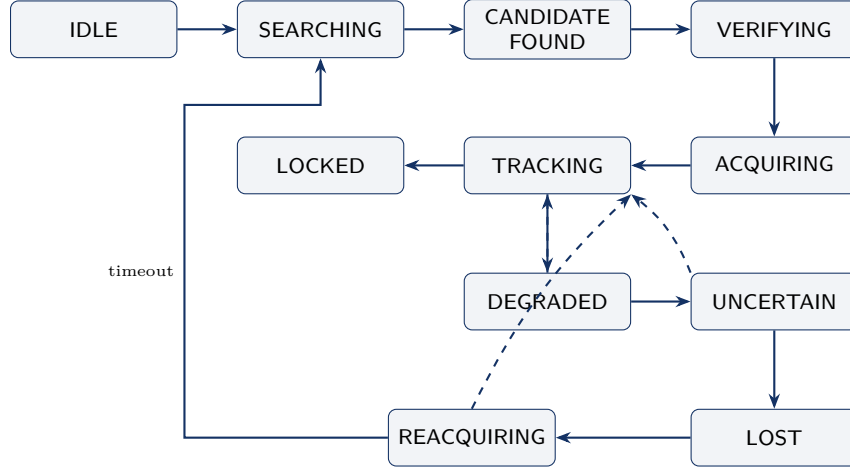


Figure 4: PAT acquisition FSM. Solid arrows: nominal progression; dashed: recovery paths (reacquisition or signal recovery). Guard conditions are given in Table 3.

State	Description	Entry condition
IDLE	Standby before simulation start	System power-on
SEARCHING	Systematic scan across the FOR	No candidate detected
CANDIDATE_FOUND	Initial optical candidate detected	Single low-confidence detection
VERIFYING	M-of-N persistence confirmation	Candidate persists across 3+ frames
ACQUIRING	Confirmed signal; slewing to beacon	Verification complete
TRACKING	Continuous closed-loop tracking	Target centred in FOV
LOCKED	High-precision lock (error < 1 mrad)	Error below lock threshold
DEGRADED	Reduced signal quality (scintillation)	Detection confidence drops significantly
UNCERTAIN	Temporary detection loss; coasting	Target undetected for several frames
LOST	Target lost beyond timeout	UNCERTAIN exceeds timeout
REACQUIRING	Predictive search near last known position	LOST entered

Table 3: FSM states and entry conditions. The source design refers to this as a “10-state” machine; eleven states are enumerated here, counting IDLE as the power-on state.

(a smooth mapping, so that errors inside the tolerance produce zero command and the lock is stable). The command is

$$u_k = \text{sat}\left(K_p \tilde{e}_k + K_i I_k + K_d \dot{\tilde{e}}_k^f; \pm u_{\max}\right), \quad I_k = \text{clip}(I_{k-1} + \tilde{e}_k \Delta t; \pm I_{\max}), \quad (19)$$

where the integrator is clamped (anti-windup) and *bleeds down by 5 % per step* while the error is inside the dead-zone. The derivative is passed through an exponential low-pass filter with $\alpha = 0.8$ to suppress pixel-quantisation noise,

$$\dot{\tilde{e}}_k^f = \alpha \dot{\tilde{e}}_{k-1}^f + (1 - \alpha) \frac{\tilde{e}_k - \tilde{e}_{k-1}}{\Delta t}, \quad (20)$$

and $u_{\max}$ equals the gimbal motor speed limit.

6.2 Latency Compensation

The gimbal path includes a `ControlDelayQueue` of $n_d \in \{1, 2, 3\}$ frames. Commands issued at frame k are applied at frame $k + n_d$. The PID error is therefore computed against the predicted position of Eq. (14) with $H \approx n_d$ (or more), so the controller acts on where the beacon *will be* when the command takes effect.

6.3 Search Strategies

- **Spiral scan:** an Archimedean spiral $r(\phi) = r_0 + b\phi$ expanding from the anchor.
- **Raster scan:** boustrophedon (lawnmower) sweep of the FOR.
- **Predictive search:** an expanding spiral centred on the Kalman-extrapolated last known position, used during reacquisition.

6.4 Dynamic Safe-FOV and Risk Assessment

The Safe-FOV engine computes an adaptive interior margin of the sensor as a function of target velocity, platform jitter amplitude, and proximity to the sensor boundary. It classifies the loss-of-lock risk as STABLE, WARNING, LOCK AT RISK, or CRITICAL and renders the corresponding box in the HUD.

6.5 Fine-PAT Handoff Readiness

The handoff engine scores readiness for transferring control to a Fine-PAT subsystem (e.g. an FSM) using five weighted criteria (Table 4):

$$\mathcal{H} = 100 (0.30 h_e + 0.25 h_j + 0.20 h_\ell + 0.15 h_v + 0.10 h_m) \%, \quad (21)$$

where each $h_\bullet \in [0, 1]$ expresses how well the criterion is satisfied. The result is mapped to one of three categorical states: READY, STABILIZING, or NOT READY.

Criterion	Threshold	Weight
Instantaneous pointing error	< 2.0 mrad	30 %
Rolling RMS jitter	< 1.5 mrad	25 %
Lock persistence	> 80 %	20 %
Angular velocity norm	< 12 mrad/s	15 %
Safe-FOV margin stability	stable	10 %

Table 4: Handoff readiness criteria.

7 Test Methodology

7.1 Unit and Integration Tests

The project includes an automated `pytest` suite covering every major subsystem (Table 5). Tests run from the project root:

```
py -m pytest tests/ -v
```

7.2 Scenario-Based Integration Testing

7.3 Multi-Algorithm Benchmarking

7.4 Adversarial Stress Testing (PAT-RED)

The `SurvivalEnvelope` module builds a 2D capability matrix mapping lock-retention percentage across target slew rates (5–40 mrad/s) and turbulence levels. The adversarial RED engine applies random parameter combinations to discover failure boundaries that hand-designed scenarios miss.

Test file	Coverage
<code>test_coordinates.py</code>	Coordinate conversions (deg $\leftrightarrow$ rad, projection)
<code>test_classical_detector.py</code>	Classical CV detector on synthetic frames
<code>test_yolo_detector.py</code>	AI detector initialisation and fallback
<code>test_fusion.py</code>	Multi-modal detection fusion logic
<code>test_kalman.py</code>	Kalman predict/update/gating
<code>test_particle_and_hybrid.py</code>	Particle filter and hybrid switching
<code>test_pid.py</code>	PID dead-zone and anti-windup
<code>test_state_machine.py</code>	FSM transitions and timeouts
<code>test_search_strategies.py</code>	Spiral and raster pattern generation
<code>test_disturbances.py</code>	Noise, vibration, turbulence, blur models
<code>test_latency_and_occlusion.py</code>	Transport delay queue and occluder physics
<code>test_safe_fov_and_handoff.py</code>	Safe-FOV risk assessment and handoff scoring
<code>test_closed_loop_mvp.py</code>	End-to-end closed-loop integration
<code>test_benchmark_and_report.py</code>	Benchmark execution and HTML report generation
<code>test_survival_envelope.py</code>	Capability envelope matrix generation
<code>test_logger.py</code>	Telemetry CSV/JSON writer
<code>test_metrics.py</code>	Metric computation correctness
<code>test_beacon_motion.py</code>	Trajectory sampling and beacon kinematics
<code>test_hybrid_tracking.py</code>	Hybrid KF $\leftrightarrow$ PF mode transitions
<code>test_target_loss_reacquisition.py</code>	Loss detection and reacquisition
<code>test_gui.py</code>	GUI widget construction (import-level)
<code>test_dataset_generator.py</code>	Synthetic training-dataset generation

Table 5: Automated test files and their coverage.

Scenario	Config	Target dynamics	Disturbances
Easy	<code>easy.json</code>	Slow circular, 1°/s	Minimal
UAV	<code>uav.json</code>	Sinusoidal, 1.5°/s	12 % noise, 15 % vibration, 15 % turbulence
Satellite	<code>satellite.json</code>	Linear, 2°/s	8 % noise, 10 % vibration
Extreme	<code>extreme.json</code>	Maneuver, 1.8°/s + distractors	20 % noise, 20 % vibration, 25 % turbulence, 15 % blur

Table 6: Predefined integration scenarios.

8 Performance Analysis

8.1 Demonstration-Tour Results

Results from the automated five-act demonstration (870 frames, 29 s simulated) are summarised in Table 8.

Interpretation. The aggregate RMSE is dominated by the initial blind search (Act 1, error $> 7^\circ$) and the occlusion-recovery phase (Act 4); steady-state locked tracking error is significantly lower. The low lock-retention figure reflects that the “locked” criterion (< 1 mrad) is strict and that the tour deliberately includes search, turbulence, and blackout phases.

8.2 Per-Act Breakdown

8.3 Algorithm Comparison

Running `py demo.py -benchmark` compares the baseline (Algorithm A) with the full pipeline (Algorithm E). Values are approximate, as reported by the benchmark suite.

Algorithm	Detector	Estimator	Prediction	Search
A	Classical CV	Direct PID	None	Raster
B	AI YOLO	Direct PID	None	Raster
C	AI YOLO	Kalman + PID	None	Spiral
D	Hybrid fusion	Kalman + PID	None	Spiral
E	Hybrid fusion	Kalman + PID	5-step lead-ahead	Spiral + predictive

Table 7: Benchmark algorithm configurations (progressive ablation from A to E).

Algorithm 1 Survival-envelope evaluation

```

1: for each slew rate  $\omega \in [5, 40]$  mrad/s do
2:   for each turbulence level  $\tau$  do
3:     Configure scenario( $\omega, \tau$ ) and run closed-loop simulation
4:      $LR(\omega, \tau) \leftarrow$  lock-retention percentage
5:   end for
6: end for
7: RED phase: sample random disturbance combinations; record configurations with loss of lock
8: return capability matrix LR and failure boundary

```

8.4 Key Observations

1. **Lead-ahead prediction** is the single most impactful component for reducing error on dynamic targets.
2. The **hybrid KF $\leftrightarrow$ PF estimator** degrades gracefully under severe turbulence without manual tuning.
3. **Dead-reckoning coasting** maintains ballistic extrapolation through occlusion, with no search penalty when the target re-emerges.
4. The **multi-state FSM** eliminates false acquisition and Track/Lost chattering common in simpler three-state machines.

8.5 Limitations

- Reported figures come from a single demonstration tour and one benchmark configuration; confidence intervals over repeated, seeded runs are not yet included.
- Simulation throughput (3.6 FPS headless) is well below real-time for the full render path.
- Turbulence is modelled as a 2D displacement field rather than a wave-optics phase screen, so scintillation statistics are approximate.
- Coarse-PAT alignment is evaluated only in angular space; range, link budget, and beam divergence are not modelled.

9 Future Improvements

9.1 Short Term

1. **EKF/UKF:** replace the linear KF to model non-linear gimbal kinematics and target acceleration.

Metric	Value
Total simulated frames	870
Root-mean-square error (RMSE)	1.37° (23.9 mrad)
Mean pointing error	0.90° (15.8 mrad)
Lock-retention rate (all acts)	12.8 %
Acquisition time (Act 1)	< 2.33 s
Average detector latency	17.69 ms
Target-loss events	0
Computation speed (headless, rendering on)	3.6 FPS

Table 8: Aggregate KPIs of the demonstration tour.

Act	Scenario	Steady-state error	Lock	Notable
1 — Acquisition	Blind search, target at +7.2°	0.034–0.059° once acquired	LOCKED at $t \approx 2.67$ s	Handoff score reached 96.2 %
2 — Dynamic tracking	2.2°/s sinusoid	0.55–1.30°	Continuous TRACKING	Lead-ahead compensates lag
3 — Turbulence	35 % turbulence, 30 % vibration	0.70–1.00°	Continuous TRACKING	Stayed on KF (severity below PF threshold)
4 — Occlusion	2.3 s total blackout	0.35–0.46° post-recovery	REACQUIRING → TRACKING	PF engaged during blackout; KF recovered
5 — Handoff	Gentle trajectory, low disturbance	0.15–0.35°	Continuous TRACKING	Error damped; handoff threshold marginal

Table 9: Per-act performance.

2. **GPU inference:** TensorRT or CUDA-backed ONNX Runtime for sub-5 ms detection latency.
3. **Multi-target tracking:** data association via JPDA or the Hungarian algorithm.
4. **Training pipeline:** end-to-end synthetic dataset generation and YOLO training from the simulator’s virtual camera.

9.2 Medium Term

5. **RL controller:** PPO or SAC agent for adaptive gain scheduling.
6. **Wave-optics propagation:** split-step Fourier Kolmogorov phase screens for physically accurate scintillation.
7. **Hardware-in-the-loop:** real gimbal motors and a machine-vision camera.
8. **3D extension:** full Euclidean geometry with range estimation for link-budget analysis.

9.3 Long Term

9. **Fine-PAT co-simulation:** integrate a Fast Steering Mirror model for complete Coarse+Fine closed-loop simulation.
10. **Constellation networking:** multi-node optical mesh simulation with dynamic link switching and handover.

Scenario	Classical+PID (A)	Hybrid+KF+Pred (E)	Improvement
Easy	$\sim 0.8^\circ$	$\sim 0.3^\circ$	$\sim 2.7\times$
UAV	$\sim 1.2^\circ$	$\sim 0.5^\circ$	$\sim 2.4\times$
Extreme	$\sim 2.0^\circ$	$\sim 0.8^\circ$	$\sim 2.5\times$

Table 10: RMSE of baseline vs. full pipeline.

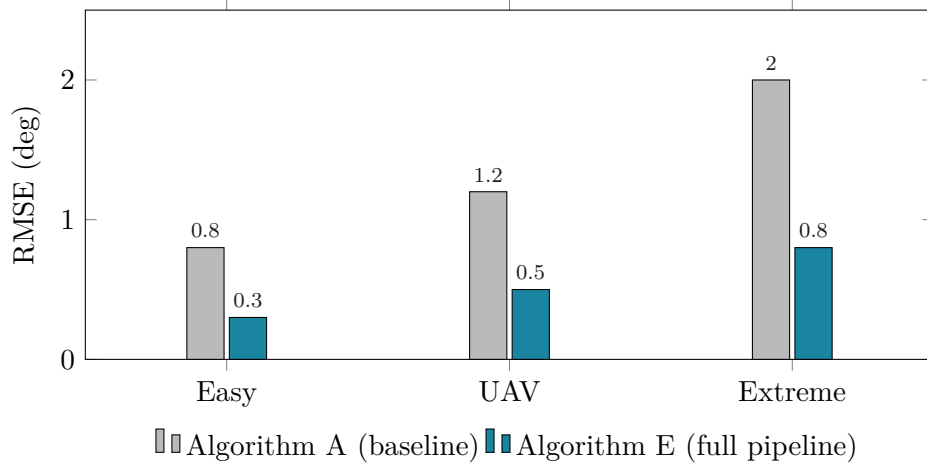


Figure 5: Approximate RMSE per scenario for the baseline and full pipeline.

10 Conclusion

This work delivers a comprehensive software platform for simulating and evaluating AI-assisted FSOC beacon tracking. It integrates classical vision, deep-learning detection, hybrid Bayesian estimation, predictive closed-loop control, and realistic disturbance modelling in one simulator. The principal technical contributions are:

- an **adaptive hybrid Kalman↔Particle filter** that handles the transition between Gaussian and non-Gaussian noise regimes;
- a multi-state **acquisition FSM** with M-of-N confirmation and hysteresis that eliminates state chattering;
- a **5-factor Fine-PAT handoff readiness engine** that quantifies coarse-alignment adequacy before beam transfer;
- an **automated benchmark suite** and **adversarial stress-testing engine** for rigorous validation.

The modular architecture allows any subsystem to be tested, extended, or replaced independently, providing a foundation for research toward real-world FSOC PAT systems.

A Notation

B Glossary

FSOC Free-Space Optical Communication.

PAT Pointing, Acquisition, and Tracking.

FOR / FOV Field of Regard / Field of View.

Symbol	Meaning
$\mathbf{x}, \mathbf{P}$	Filter state and covariance
$\mathbf{F}, \mathbf{H}, \mathbf{Q}, \mathbf{R}$	Transition, measurement, process- and measurement-noise matrices
$\mathbf{y}, \mathbf{S}, \mathbf{K}$	Innovation, innovation covariance, Kalman gain
N, N_{eff}	Particle count, effective sample size
S	Optical severity score
κ	Contour circularity
H	Lead-ahead horizon (frames)
δ, α	PID dead-zone width, derivative filter coefficient
C_n^2	Refractive-index structure constant

FSM	Finite State Machine (or Fast Steering Mirror, depending on context).
KF / PF	Kalman Filter / Particle Filter.
SIR	Sequential Importance Resampling.
DWPA	Discrete Wiener Process Acceleration.
NMS	Non-Maximum Suppression.
LOS	Line of Sight.

References

- [1] H. Kaushal and G. Kaddoum, “Optical Communication in Space: Challenges and Mitigation Techniques,” *IEEE Communications Surveys & Tutorials*, vol. 19, no. 1, pp. 57–96, 2017.
- [2] A. Carrasco-Casado *et al.*, “Free-Space Optical Links for Space Communication Networks,” in *Springer Optical Networks*, 2021.
- [3] L. C. Andrews and R. L. Phillips, *Laser Beam Propagation through Random Media*, 2nd ed., SPIE Press, 2005.
- [4] R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Trans. ASME Journal of Basic Engineering*, vol. 82, pp. 35–45, 1960.
- [5] N. J. Gordon, D. J. Salmond, and A. F. M. Smith, “Novel Approach to Nonlinear/Non-Gaussian Bayesian State Estimation,” *IEE Proceedings F*, vol. 140, no. 2, pp. 107–113, 1993.
- [6] J. Redmon and A. Farhadi, “YOLO: Real-Time Object Detection,” in *Proc. IEEE CVPR*, 2016.
- [7] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” <https://github.com/ultralytics/ultralytics>, 2023.